國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

Ching-I Huang

Multi-class Classification (4.2 and 4.3.4)

- The overall likelihood function

$$P(\mathbf{y}|\mathbf{w}_1, b_1, ..., \mathbf{w}_K, b_K) = \prod_{n=1}^{N} \prod_{k=1}^{K} P(\mathcal{C}_k|\mathbf{x}_n)^{y_{nk}}$$

- Posterior Probability

$$y_{kn} = P(C_k \mid x_n, \mathbf{w})$$

For a given input x_n ,the model predicts the probability of each class.

- Likelihood Function

$$L(\mathbf{w}) = \prod_{n=1}^{N} \prod_{k=1}^{K} P(C_k \mid x_n, \mathbf{w})^{t_{kn}}$$

With a given dataset, find parameters ***w*** such that the model's posterior probabilities best match the target. (maximize the product of all posteriors over the dataset)

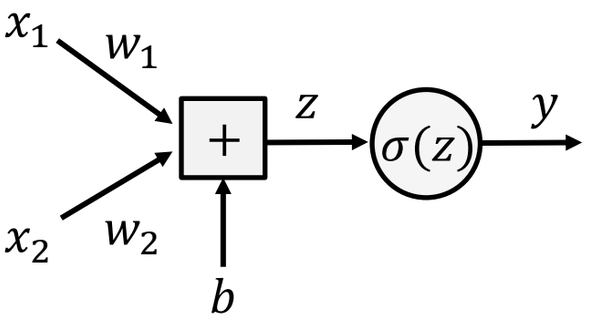
Concept of Neuron Network

- Limitation of logistic regression

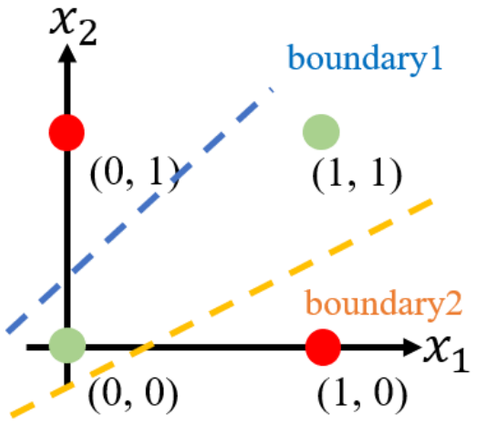
XOR

$$x = \left\{ \begin{bmatrix}0\\0\end{bmatrix}_A \begin{bmatrix}0\\1\end{bmatrix}_B \begin{bmatrix}1\\0\end{bmatrix}_C \begin{bmatrix}1\\1\end{bmatrix}_D \right\}$$

$$y = f^*(x) \quad 0 \quad 1 \quad 1 \quad 0$$



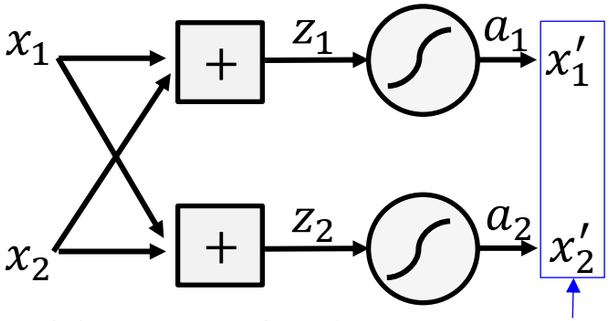
2-D input data x_1, x_2



Inseparable!

The boundary can be modified through adjusting the weighting w_1 and w_2, but it cannot classify this case.

- **Feature transformation**



$$\sigma(z) = max\{0, z\}$$

$$f(x; w_1, w_2, b_1, b_2) = w_2^T max\{0, w_1^T + b_1\} + b_2$$

$$w_1 = \begin{bmatrix}1 & 1\\1 & 1\end{bmatrix} \cdot b_1 = \begin{bmatrix}0\\-1\end{bmatrix} \quad w_2 = \begin{bmatrix}1\\-2\end{bmatrix} \quad b_2 = 0$$

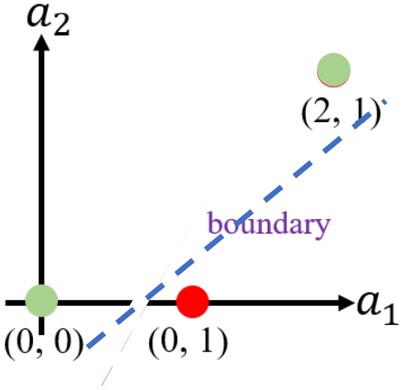
w_1 x b_1 $z^{(1)}$ maxout w_2 $a^{(1)}$

$$\text{A: } \begin{bmatrix}z_1^{(1)}\\z_2^{(1)}\end{bmatrix} = \begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix} + \begin{bmatrix}0\\-1\end{bmatrix} = \begin{bmatrix}0\\-1\end{bmatrix} \rightarrow \begin{bmatrix}0\\0\end{bmatrix} \quad y = \begin{bmatrix}1 & -2\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix} = 0$$

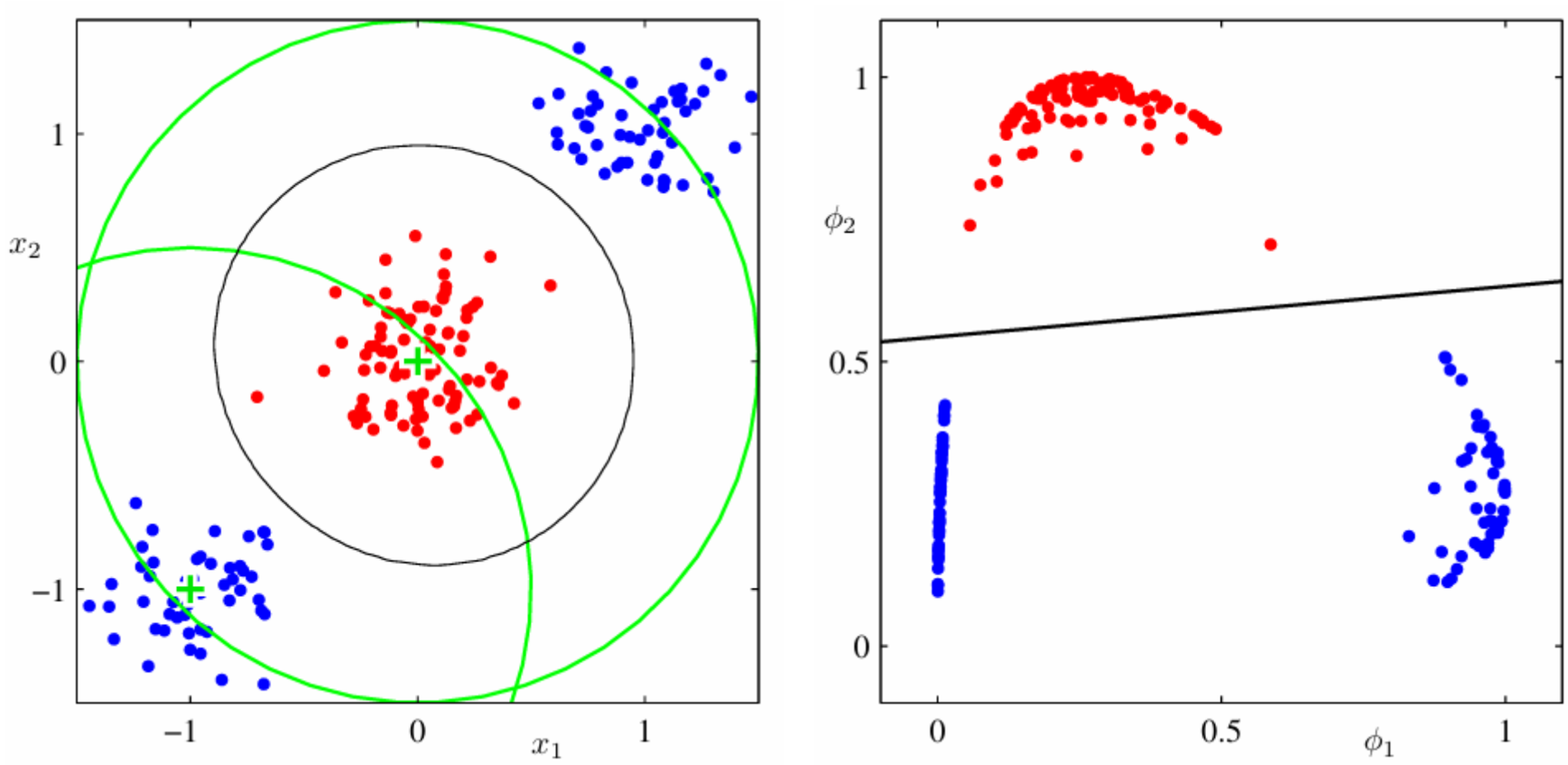
$$\text{B: } \begin{bmatrix}z_1^{(1)}\\z_2^{(1)}\end{bmatrix} = \begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} + \begin{bmatrix}0\\-1\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix} \rightarrow \begin{bmatrix}1\\0\end{bmatrix} \quad y = \begin{bmatrix}1 & -2\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = 1$$

$$\text{C: } \begin{bmatrix}z_1^{(1)}\\z_2^{(1)}\end{bmatrix} = \begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0\\-1\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix} \rightarrow \begin{bmatrix}1\\0\end{bmatrix} \quad y = \begin{bmatrix}1 & -2\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = 1$$

$$\text{D: } \begin{bmatrix}z_1^{(1)}\\z_2^{(1)}\end{bmatrix} = \begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}\begin{bmatrix}1\\1\end{bmatrix} + \begin{bmatrix}0\\-1\end{bmatrix} = \begin{bmatrix}2\\1\end{bmatrix} \rightarrow \begin{bmatrix}2\\1\end{bmatrix} \quad y = \begin{bmatrix}1 & -2\end{bmatrix}\begin{bmatrix}2\\1\end{bmatrix} = 0$$

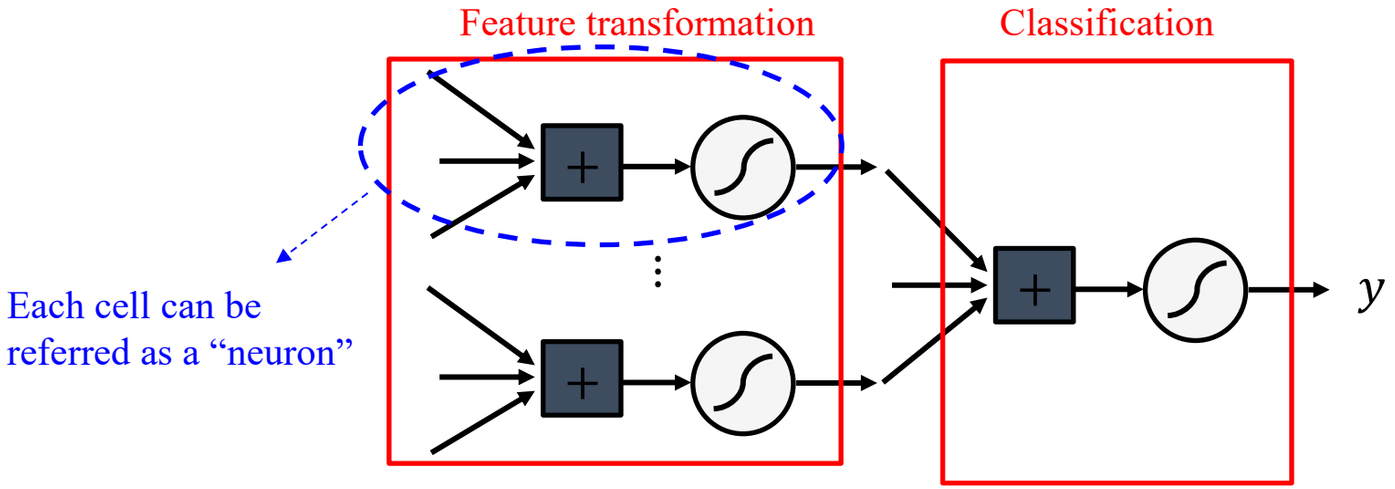


Concept of Neuron Network

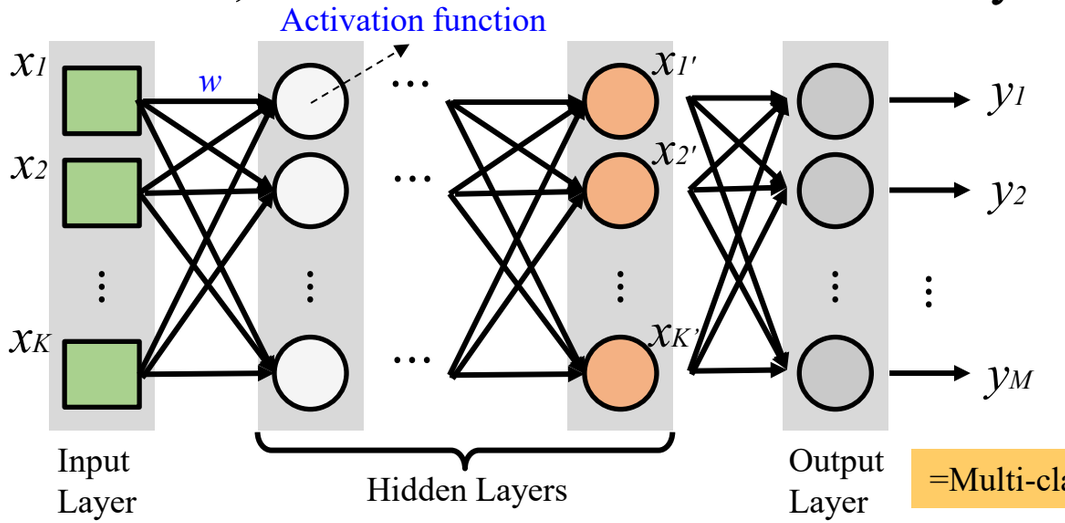


Concept of Neuron Network

- Cascading logistic regression models to achieve binary classification



- In multi-class classification, feature transformation is usually multiple layers (Hidden layers).



The weighting of the neural network will be iteratively updated by gradient descent method.

The number of the output classes *M* is determined by the training data.

=Multi-class Classifier (Softmax)

Deep Neuron Network

- Goal: To approximate a mapping function f^*. A feedforward network defines a mapping $y = f(x;\theta)$ and learns the value of the parameters θ that result in the best function approximation.
- The basic neural network model, which can be describeda series of functional transformations. Or instance, three functions $f(1)$, $f(2)$, and $f(3)$ can form a chain as $f(x) = f(3)(f(2)(f(1)(x)))$.

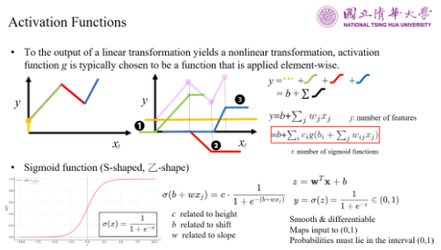
Suppose that we choose a linear model, with θ consisting of bias b and weight w.

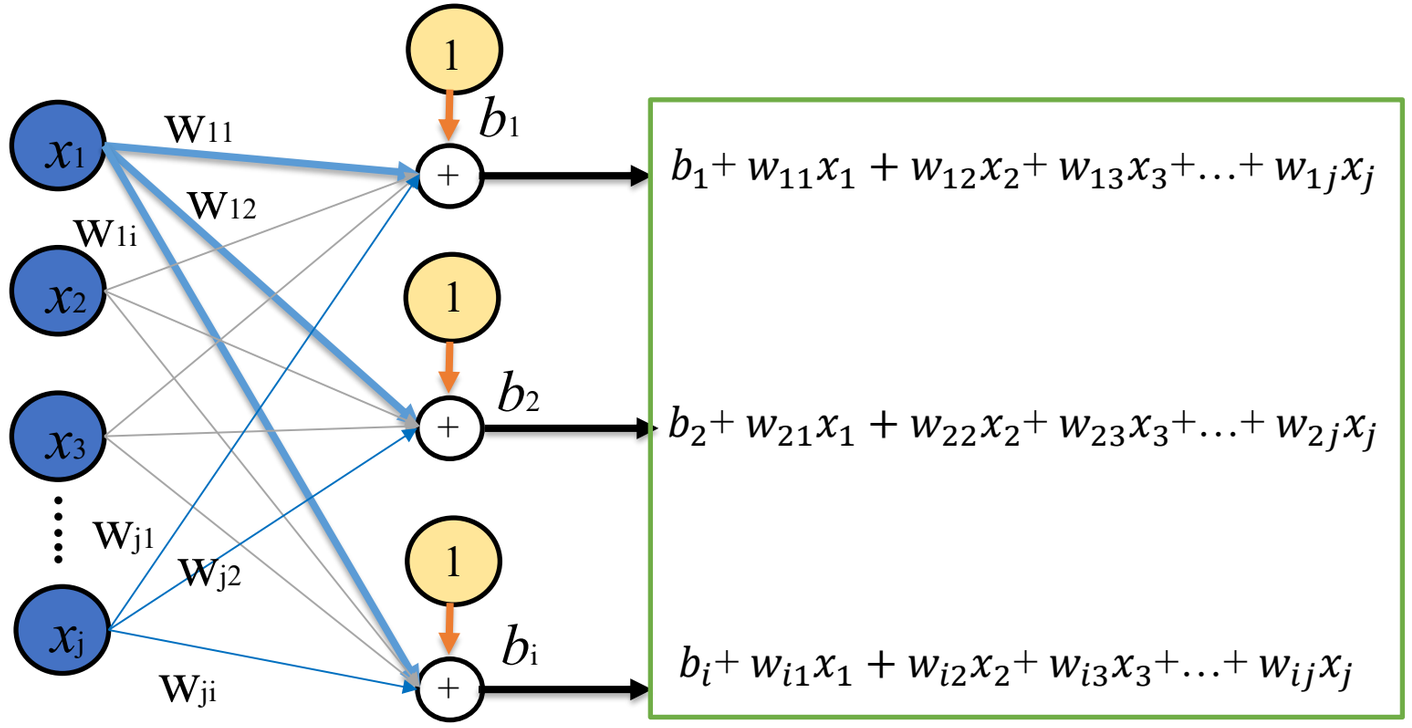
$$y = b + \sum_j w_j x_j$$ j: number of features

$$= b + \sum_i c_i g\left(b_i + \sum_j w_{ij} x_j\right)$$

i: number of sigmoid functions

$$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_j \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & w_{13} & \dots & w_{1j} \\ w_{21} & w_{22} & w_{23} & \dots & w_{2j} \\ w_{31} & w_{32} & w_{33} & \dots & w_{3j} \\ w_{i1} & w_{i2} & w_{i3} & \dots & w_{ij} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_j \end{bmatrix}$$



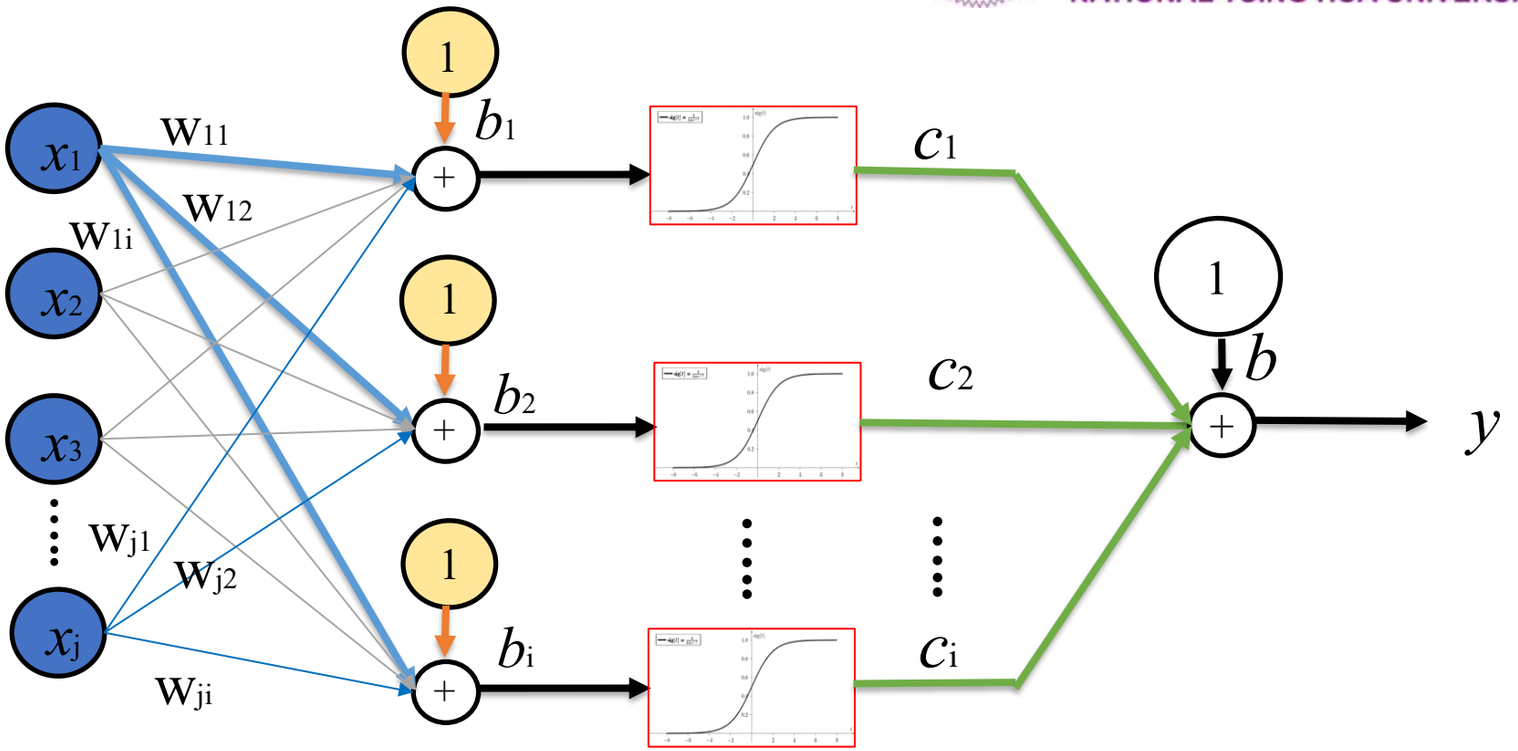


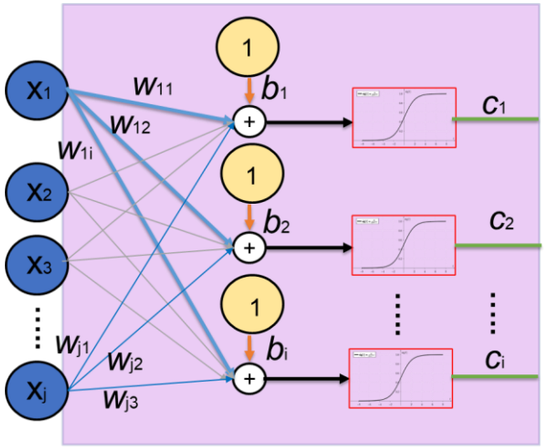
Deep Neuron Network

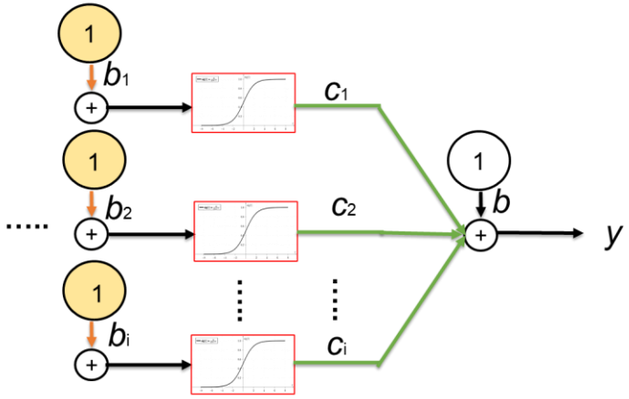
$$y = b + \sum_i c_i \, g\left(b_i + \sum_j w_{ij} x_j\right)$$

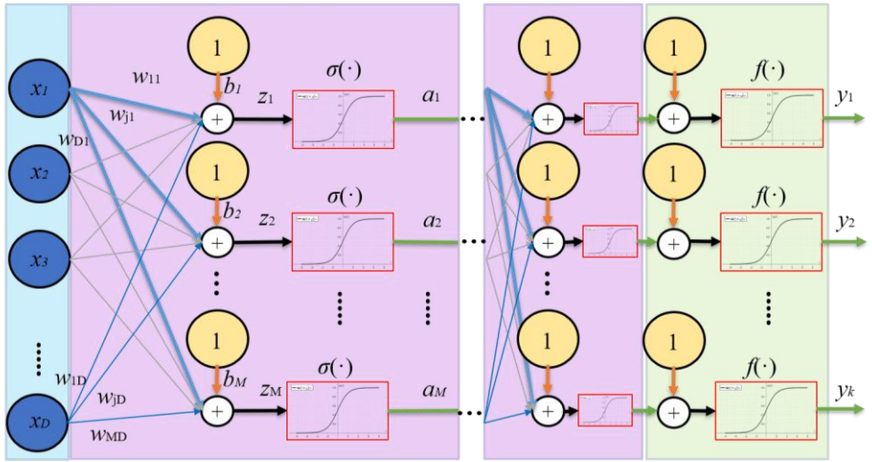
$$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_j \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & w_{13} & \dots & w_{1j} \\ w_{21} & w_{22} & w_{23} & \dots & w_{2j} \\ w_{31} & w_{32} & w_{33} & \dots & w_{3j} \\ w_{i1} & w_{i2} & w_{i3} & \dots & w_{ij} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_j \end{bmatrix}$$

$$y = b + C^T \, g(B_{i\times 1} \, W_{i\times j} \, x_{1\times j})$$



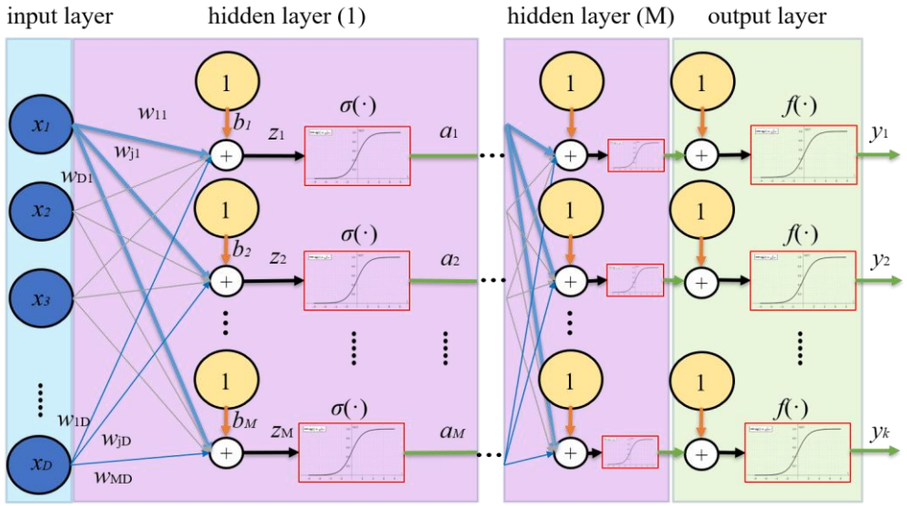






Perceptron, Neuron

Deep Neuron Network



A. *Hidden layer*

First we construct M linear combinations of the input variables $x_1, ..., x_i, ..., x_D$ in the form

$$z_j = \sum_{i=1}^{D} w_{ji}^{(h)} x_i$$

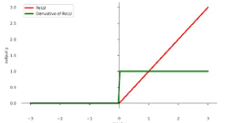
where the neural unit $j = 1, ..., M$, and the superscript (h) indicates that the corresponding parameters are in the h^{th} 'layer' of the network. The quantities a_j are known as *activations*. Each of them is then transformed using a

Hidden unit: 激勵函數

Nonlinear functions like the logistic sigmoid or the hyperbolic tangent (tanh) are commonly used as activation functions. Sigmoidal units tend to saturate when z is either very positive or negative, which can hinder gradient-based learning. The hyperbolic tangent, resembling the identity function more closely with $tanh(0) = 0$(compared to $\sigma(0) = 1/2$, generally performs better than the logistic sigmoid. However, the widespread saturation (small ∇E) would slow down learning speed. Therefore, many modern neural networks use rectified linear units (ReLU) to avoid these issues while retaining essential properties.

- **ReLU**:

$$\sigma(\cdot) = max\{0, z\}$$



ReLU is not differentiable at $z = 0$; software implementations use a one-sided derivative. ReLU outputs zero for half of its domain, allowing its derivatives to remain large and consistent where active. Since the second derivative is always zero, the gradient direction remains more useful for learning compared to activation functions that introduce second-order effects.

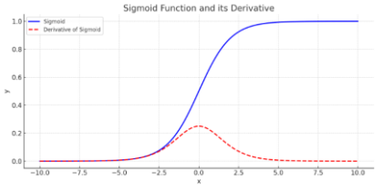
- **Maxout units**: Maxout units generalize rectified linear units (ReLU) further. Each maxout unit divides its input z into groups of k values, and then outputs the maximum element from one of these groups.

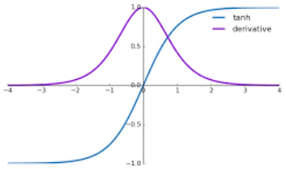
$$\sigma(\mathbf{z})_i = max_{j \in \mathbb{G}^i} \; z_j$$

where $\mathbb{G}^i$ is the indices of the inputs for group i, $\{(i-1)k+1, ..., ik\}$. Maxout units can thus be seen as **learning the activation function** itself rather than just the relationship between units. Different examples may update different parts of the network, then, offering some "redundancy" that helps to resist the catastrophic forgetting phenomenon. Each maxout unit is now parametrized by multiple weight vecotors instead of one, and implies it require more training data.

B. *Output layer*

The final layer of a feedforward network is called the output layer. The output activations $f(\cdot)$ are transformed by an activation function to produce network outputs. The choice of activation function depends on the data type and the assumed target distribution.





Deep Neuron Network

Figure 5.4 Example of the solution of a simple two-class classification problem involving synthetic data using a neural network having two inputs, two hidden units with 'tanh' activation functions, and a single output having a logistic sigmoid activation function. The dashed blue lines show the $z = 0.5$ contours for each of the hidden units, and the red line shows the $y = 0.5$ decision surface for the network. For comparison, the green line denotes the optimal decision boundary computed from the distributions used to generate the data.